Домашнее задание 4. Обеспечение отказоустойчивости средствами Apache ZooKeeper

Эль-Сибаи Амир, группа ИИИ-3-ИнжДан-1-2, вариант 8: «Программист, разработчик Frontend»

1. Установка

Java 17 из Homebrew (OpenJDK 17.0.16), `JAVA_HOME` задан на неё. Архив `apache-zookeeper-3.9.6-bin.tar.gz` скачал с `archive.apache.org`, контрольная сумма SHA-512 совпала с опубликованной. Распаковал его в три копии `zk1`, `zk2`, `zk3`.

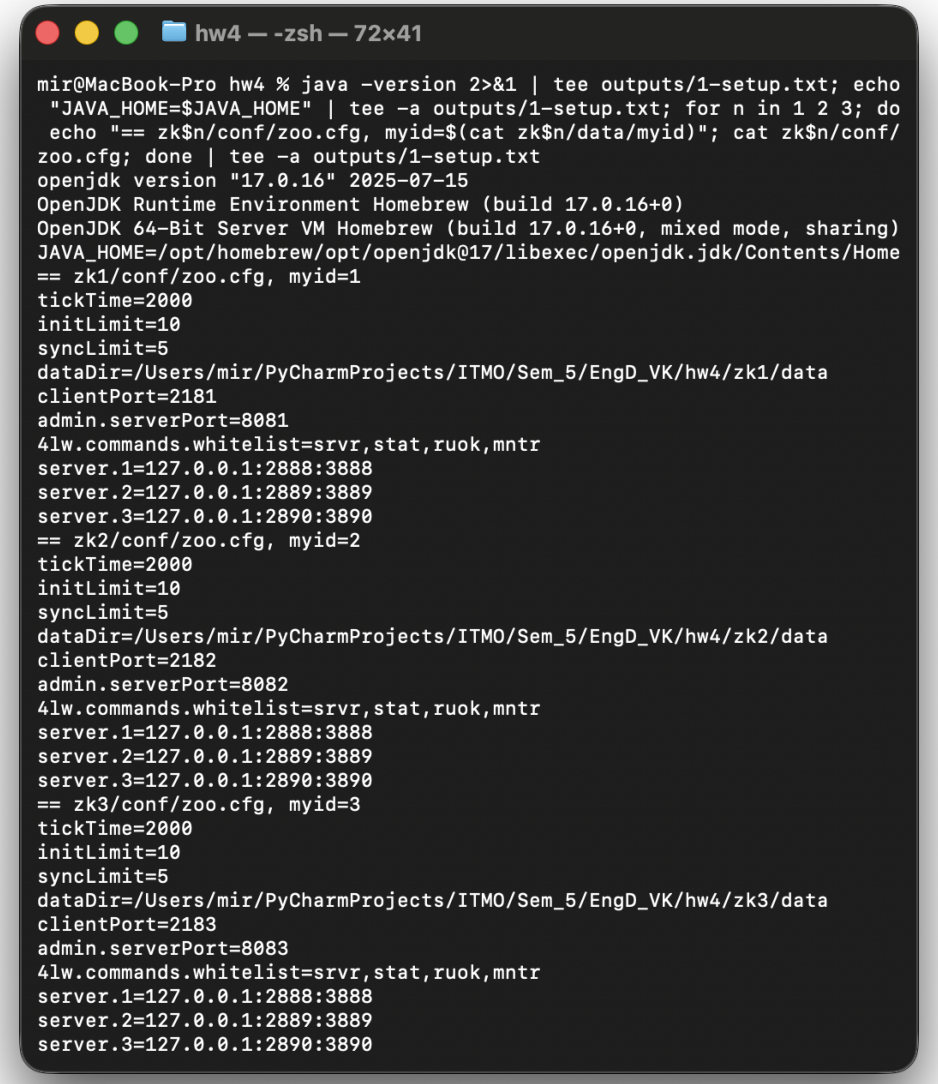


Рис. 1: Java 17, `JAVA_HOME` и конфигурации трёх серверов

2. Ансамбль ZooKeeper

Конфигурация

Строки `server.N` у всех трёх серверов одинаковые, а клиентский порт, порт встроенного веб-сервера и `dataDir` у каждого свои. Все три конфигурации — на рис. 1.

У `zk2` и `zk3` — порты 2182 и 2183, 8082 и 8083 и свои папки `data`. В каждой `data` лежит `myid` с номером 1, 2 или 3: по нему сервер узнаёт, какая строка `server.N` про него. Первый порт в строке `server.N` — для связи последователей с лидером, второй — для выбора лидера.

Запуск и роли

Запустил три сервера штатной командой `zkServer.sh start`. Лидером стал `zk2`, `zk1` и `zk3` — последователи.

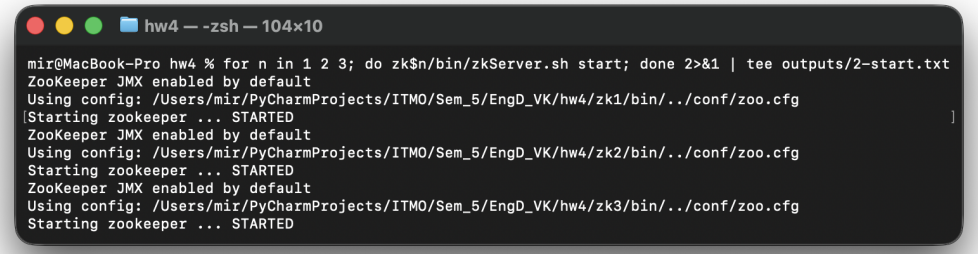


Рис. 2: Запуск трёх серверов

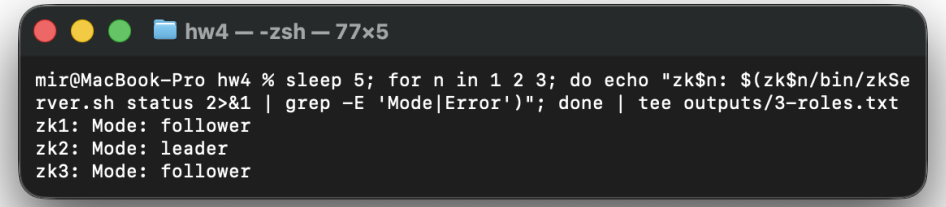


Рис. 3: Роли серверов

Потеря одного сервера

Остановил лидера `zk2`. Оставшиеся два сервера — это большинство из трёх, они выбрали нового лидера: им стал `zk3`, `zk1` остался последователем. Ансамбль работает.

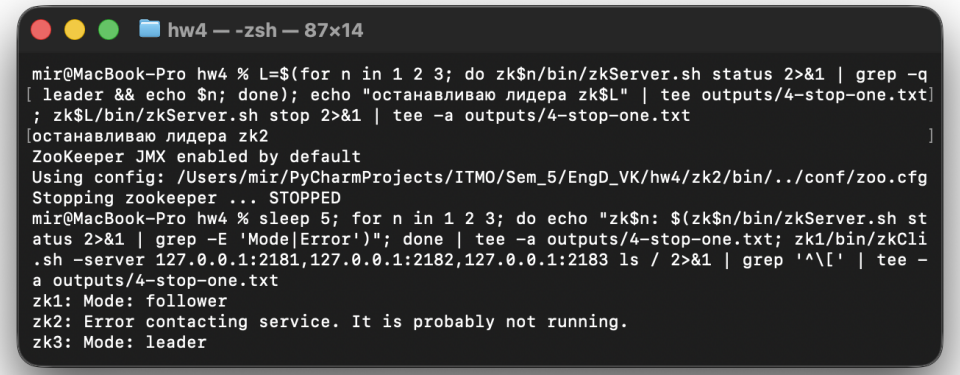


Рис. 4: Без одного сервера ансамбль работает

Потеря двух серверов

Остановил ещё `zk1`. Один `zk3` — уже не большинство. Процесс жив, но запросы не обслуживает: `zkServer.sh status` у всех трёх пишет `Error contacting service`, а на команду `srvr zk3` отвечает `This ZooKeeper instance is not currently serving requests`.

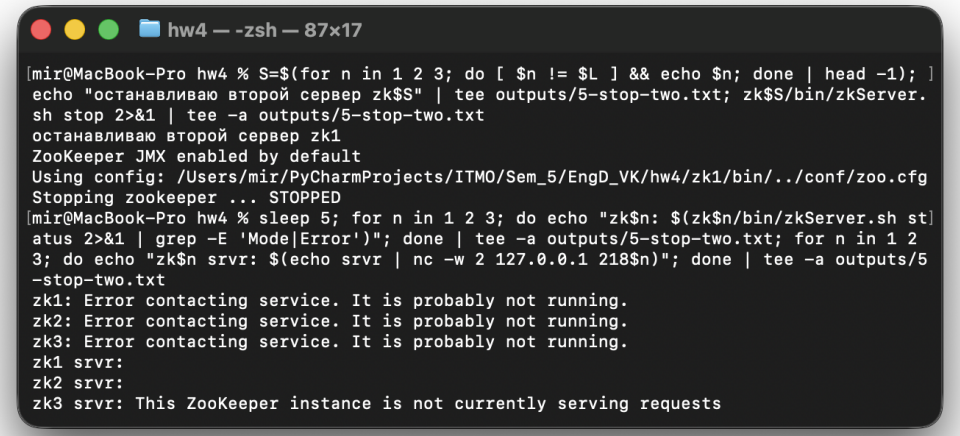


Рис. 5: Без двух серверов ансамбль не отвечает

После запуска `zk2` и `zk1` кворум вернулся: `zk3` — лидер, `zk1` и `zk2` — последователи.

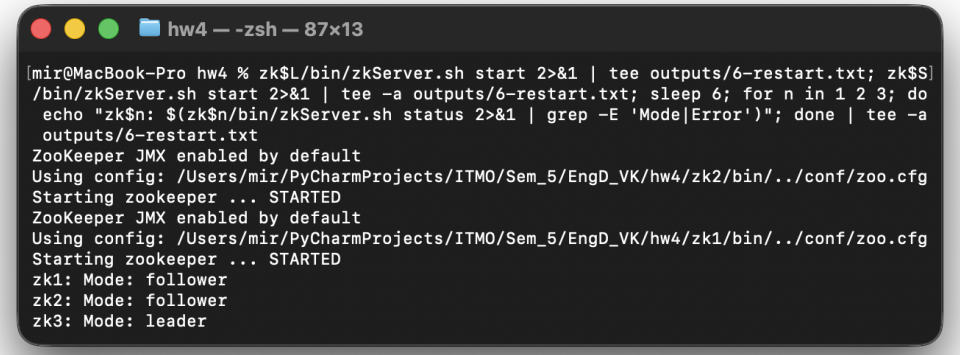


Рис. 6: Ансамбль снова из трёх серверов

Эфемерный узел

Во втором окне в `zkCli` создал эфемерный узел `create -e /demo hello`. Из первого окна `stat /demo` показал владельца `ephemeralOwner = 0x200081b23990000` — это номер сессии второго окна. Закрыл второе окно, через 30 с (тайм-аут сессии `zkCli`) `stat /demo` ответил `Node does not exist: /demo`: сессия истекла, и ZooKeeper удалил её эфемерный узел.



Рис. 7: Эфемерный узел создан во втором окне

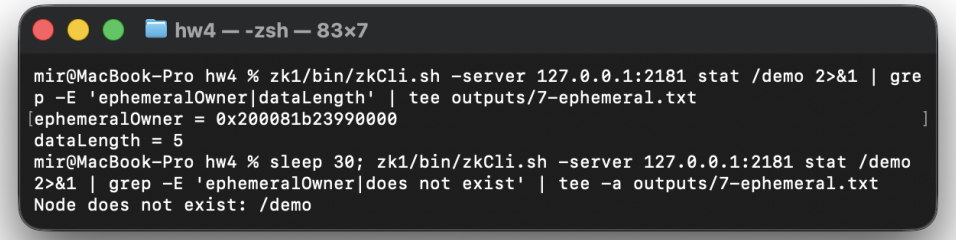


Рис. 8: После закрытия окна узел исчез

3. Координатор

`coordinator.py` — это расчёт пар навыков из ДЗ 3, обёрнутый в координацию через ZooKeeper (библиотека kazoo). Данные читаются из RustFS, из бакета `hh-lake`, частями по 8 МБ, как в ДЗ 3.

- Сессия с тайм-аутом 10 с. Если координатор не подаёт признаков жизни 10 с, ZooKeeper закрывает его сессию.
- Выбор лидера. Обе копии регистрируются эфемерными последовательными узлами в `/hw4/<запуск>/election`, лидер — тот, у кого номер меньше. Запасной ждёт, пока узел лидера исчезнет.
- Эпоха. Новый лидер перезаписывает узел `epoch`, и номер его версии становится эпохой: у первого лидера 1, у следующего 2.
- Расчёт. Файл делится на 23 задачи по 10 000 строк. После каждой задачи лидер кладёт счётчик пар в RustFS, а в узел `checkpoint` пишет номер задачи и байт, с которого начинается следующая.
- Запасной, став лидером, читает чекпоинт и продолжает со следующей задачи с этого байта.
- Публикация. Результат пишется объектом в RustFS, а узел-указатель `result` получает ссылку на него.

Без защиты указатель перезаписывается безусловно. С `FENCING=1` запись идёт транзакцией: сначала проверка, что версия `epoch` всё ещё равна моей эпохе, потом запись указателя. Если эпоха устарела, транзакция отклоняется целиком:

```
transaction = zk.transaction()
transaction.check(f"{root}/epoch", epoch)
transaction.set_data(f"{root}/result", pointer.encode())
check, stat = transaction.commit()
```

`failover.py` приостанавливает лидера сигналом `SIGSTOP`, когда тот закончил задачу 10, засекает время до смены лидера и возобновляет его сигналом `SIGCONT`, как только запасной опубликовал результат. `check.py` печатает указатель и сверяет результат с таблицей пар ДЗ 3 (`results/mapreduce/k2.csv` в `hh-lake`). Журналы координаторов и `failover.py` лежат в `logs/`.

4. Приостановка лидера без защиты

Запуск `run1`, `FENCING=0`. Лидер A приостановлен после задачи 10. Через 11,2 с лидером стал B с эпохой 2, продолжил с чекпоинта с задачи 11, записал результат, и указатель получил версию 1. Затем A возобновился: его сессия истекла (SUSPENDED, LOST, затем новая сессия CONNECTED), но он досчитал задачи 11–22.

В журналах две строки «записал результат», указатель в итоге версии 2 и ссылается на результат старого лидера A с эпохой 1. A не знал, что его сменили: пока он стоял, он не выполнял ни одной инструкции, а проснувшись, просто продолжил расчёт и записал. Оба результата совпали с ДЗ 3: пар уникальных 26 295, всего 122 143.

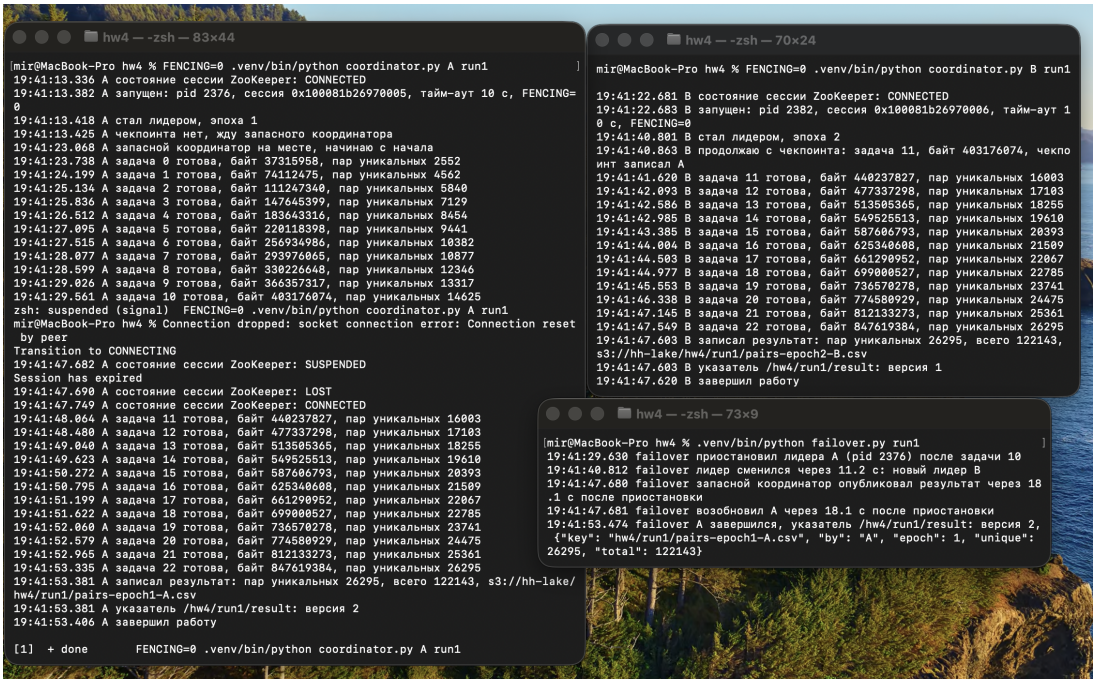


Рис. 9: Без защиты: A, B и `failover.py`

5. Приостановка лидера с защитой по эпохе

Запуск `run2`, `FENCING=1`. Он сделан первым, запуск без защиты позже. Сценарий тот же: A приостановлен после задачи 10, через 10,5 с лидером стал B с эпохой 2, продолжил с задачи 11 и записал результат, указатель получил версию 1.

Возобновивший работу A досчитал, но его запись отклонена: `BadVersionError`, эпоха 1 устарела. Указатель остался версии 1 от запасного координатора B с эпохой 2. Результат по нему совпал с ДЗ 3: пар уникальных 26 295, всего 122 143. Объект `pairs-epoch1-A.csv` в RustFS старый лидер успел записать до проверки эпохи, но на него ничего не ссылается.

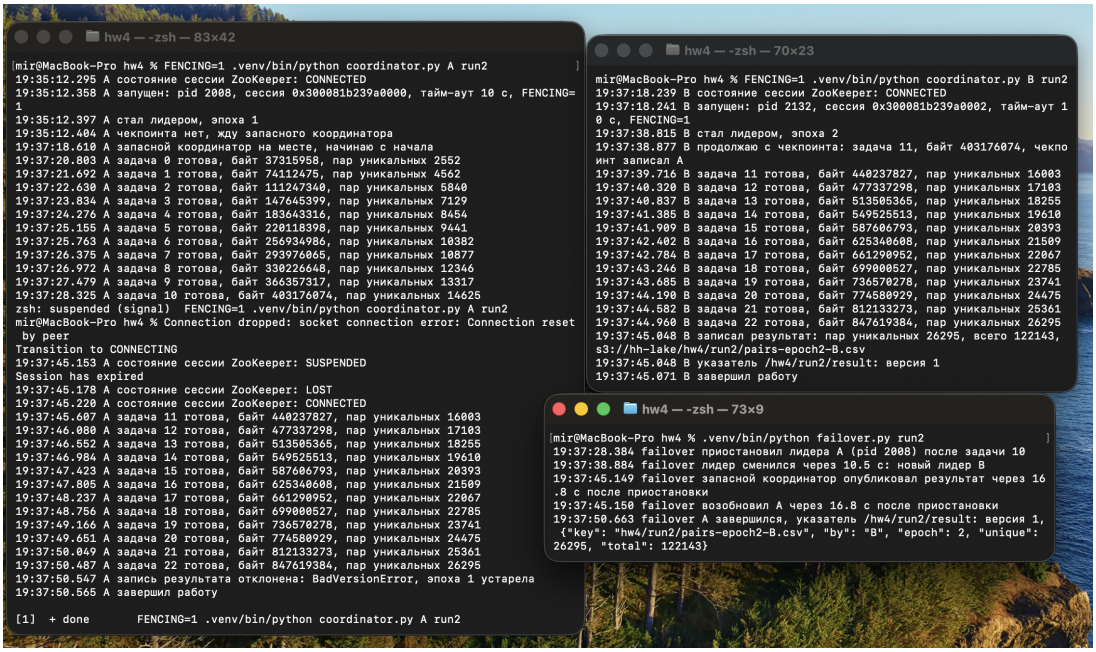


Рис. 10: С защитой: A получил `BadVersionError`, B, `failover.py`

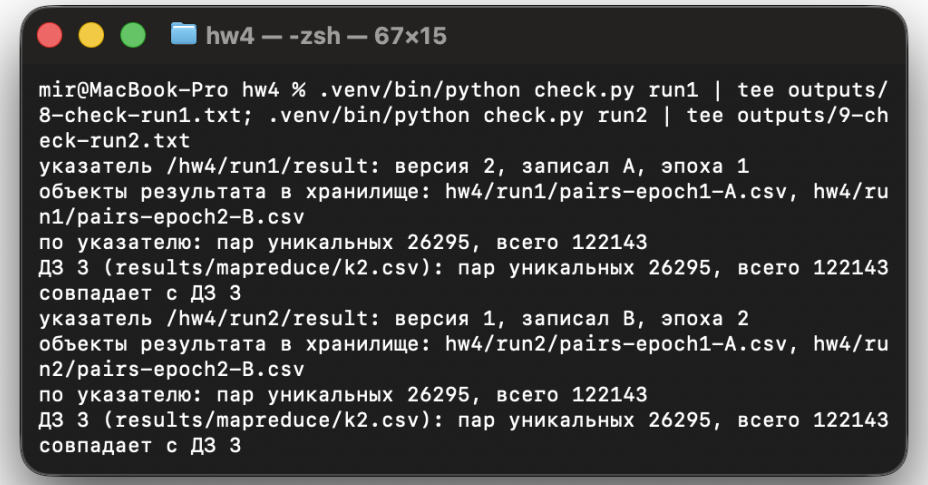


Рис. 11: Сверка обоих запусков с ДЗ 3: без защиты указатель версии 2 от A, с защитой версии 1 от B

6. Время смены лидера

Запуск	От приостановки до смены лидера	Лидер был приостановлен
`run1`, без защиты	11,2 с	18,1 с
`run2`, с защитой	10,5 с	16,8 с

Смена лидера занимает чуть больше тайм-аута сессии 10 с: ZooKeeper проверяет истёкшие сессии шагами по `tickTime` (2 с), затем удаляет эфемерный узел лидера и оповещает запасного. Лидер стоял 17–18 с, а не 40: `failover.py` возобновлял его сразу после того, как запасной закончил расчёт и опубликовал результат.

7. Ответы на вопросы

Почему серверов три, а не два и не четыре

ZooKeeper работает, только пока живо строгое большинство серверов: из N нужно $\lfloor N/2 \rfloor + 1$.

- Из двух серверов большинство — оба. Потеря любого останавливает ансамбль, и это хуже одного сервера: точек отказа стало две.
- Из трёх большинство — два, ансамбль переживает потерю одного. Это показано в разделе 2: без `zk2` он работал, без `zk2` и `zk1` — нет.
- Из четырёх большинство — три, ансамбль всё так же переживает потерю только одного. Сервер добавился, а запаса нет: каждую запись подтверждают уже три сервера, ломаться есть чему больше, а при разрыве сети 2 на 2 большинства нет ни у одной половины.

Поэтому серверов берут нечётное число: три переживают один отказ, пять — два.

Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper

ZooKeeper знает, кто лидер сейчас, но не может остановить старого лидера. Пока A стоял на паузе, его сессия истекла, и лидером стал B. Но сам A об этом не узнал: на паузе он не выполнял код, а проснувшись, продолжил с того же места. Проверка на лидера перед записью тоже не спасает: процесс может замереть сразу после проверки, а записать уже после смены лидера. Единственное место, которое видит одновременно и саму запись, и текущую эпоху, — получатель записи. Только он может атомарно сравнить эпоху из запроса с последней известной и отказать старой. В моём случае получатель — узел-указатель: эпоха проверяется в той же транзакции, что и запись. Здесь ZooKeeper выступает хранилищем, принимающим запись, а не службой выбора лидера. Будь результат в базе данных или S3, проверять эпоху пришлось бы им.